

This reading document provides a technical breakdown of the fine-tuning process implemented in script. It explains how a compact Vision-Language Model (VLM) is specialized for medical diagnostics using Parameter-Efficient Fine-Tuning (PEFT) techniques.

Project Overview: SmolVLM2 fine-tuning on Kvasir-VQA

The goal of this pipeline is to take SmolVLM2-2.2B, a general-purpose vision model, and specialize it for Gastrointestinal (GI) Endoscopy. By fine-tuning on the Kvasir-VQA dataset, the model learns to identify anatomical landmarks, pathological findings (like polyps), and clinical instruments with high precision.

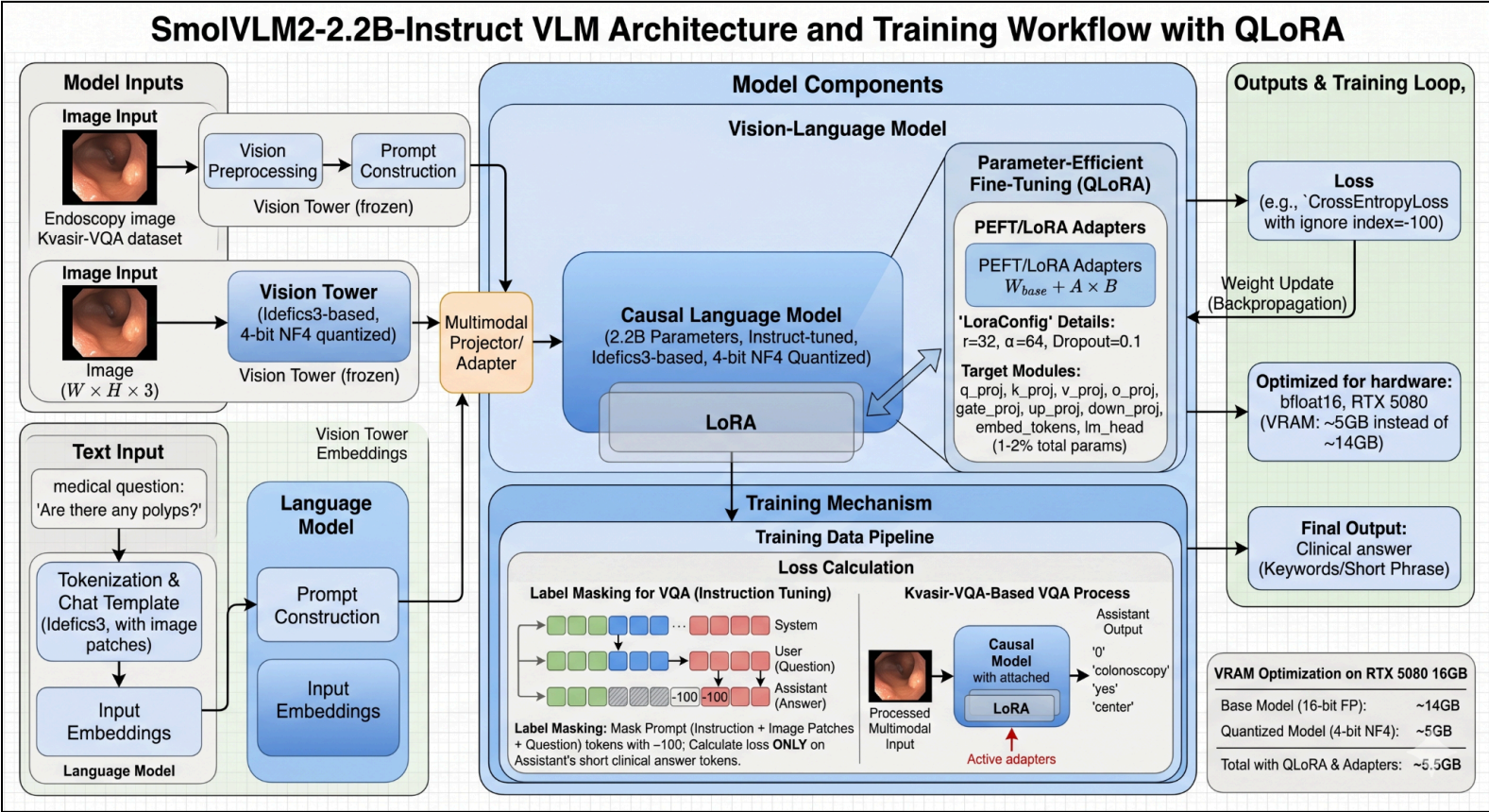


Figure 1: Block diagram for workflow of Fine-tuning process

A. The Core Architecture: QLoRA

To train a 2.2-billion parameter model on consumer hardware (like RTX 5080), the script utilizes QLoRA (Quantized Low-Rank Adaptation).

1. **4-bit Quantization:** The base model weights are compressed from 16-bit to 4-bit using the `NF4` (NormalFloat4) data type. This reduces VRAM usage by ~70% without significant loss in accuracy.
2. **LoRA Adapters:** Instead of updating all 2.2 billion weights, we inject tiny, trainable "adapter" matrices into the model. Only these adapters are updated during training.

B. Key Hyperparameters Explained

1. QLoRA Configuration (`LoraConfig`)

- Rank (`r=32`): This determines the "width" of the adapter matrices. A rank of 32 is a sweet spot for medical tasks—it provides enough capacity to learn complex endoscopic features while remaining memory-efficient.
- Alpha (`alpha=64`): A scaling factor for the LoRA weights. A common rule of thumb is to set $\alpha = 2r$. This helps stabilize the learning process.
- Target Modules: The adapters are applied to specific layers including `q_proj`, `k_proj`, and `v_proj` (Attention) as well as `up_proj` and `down_proj` (MLP). Targeting more modules generally leads to better performance, effectively mimicking "full fine-tuning."

C. The Training Workflow

Training Samples	Evaluation (Validation) Samples	Hold-out test samples
5000	100	5000

Step 1: Data Preprocessing

The model doesn't just "see" an image; it processes a sequence of tokens. The script uses a Chat Template to format the data:

- **System:** Tells the model it is a medical assistant with following prompt

Prompt for system instruction:

```
SYSTEM_INSTRUCTION = "You are a medical AI assistant specialized in gastrointestinal endoscopy. Analyze the provided endoscopic image carefully and answer the clinical question. Answer in keywords or short phrases only. Questions based on: anatomical landmarks, pathological findings, instrument presence, image quality. Examples for answers: '0', 'colonoscopy', 'none', 'yes', 'center'."
```

- **User:** Contains the `<image>` tag and the clinical question.
- **Assistant:** Contains the ground-truth answer.

Step 2: Label Masking

A crucial step in the `preprocess_vqa_no_padding` function is setting labels for the "System" and "User" tokens to -100. In the context of Large Language Models (LLMs) and training libraries like Hugging Face transformers, -100 is the standard "ignore index" for calculating loss. The value -100 tells the loss function: "Do not calculate the error for this specific token; skip it entirely. This ensures the model is only penalized (and thus only learns) based on its ability to predict the Assistant's answer, not the question it was just asked.

Step 3: Optimization & Memory Management

- **Gradient Checkpointing:** Trades compute for memory. It doesn't store all intermediate activations during the forward pass, recomputing them during the backward pass instead.
- **ClearCacheCallback:** Custom logic that clears the GPU's "trash" (fragmented memory) before evaluation to prevent `Out of Memory` (OOM) errors.
- **Batch Size & Gradient Accumulation:**
 - `per_device_train_batch_size = 1`
 - `gradient_accumulation_steps = 16`
 - Effective Batch Size = 16. This allows you to simulate a large, stable batch size even though the GPU only processes one image at a time to save VRAM.
- **Learning Rate ($2 * 10^{-4}$):** A standard starting point for QLoRA. It's high enough to learn quickly but controlled by a Cosine Scheduler that tapers the rate off toward the end of training.
- **Weight Precision (`bf16=True`):** Utilizes the `bfloat16` format supported by RTX 5080, which offers better numerical stability than standard `fp16` for deep learning.

Step 4: Saving best model

The saving of the "best" model in the script is managed by a combination of monitoring and logic parameters within the `TrainingArguments`. Instead of simply saving the model at the very end of training, the `Trainer` evaluates performance at specific intervals and keeps the version that performed best.

Here is the step-by-step breakdown of how that happens:

1. The Evaluation Trigger

- `eval_strategy="steps" & eval_steps=50`: Every 50 training steps, the `Trainer` pauses the training process to run your 100-sample evaluation dataset through the model.
- `save_strategy="steps" & save_steps=50`: This ensures that a checkpoint is saved at the same time the evaluation happens.

2. Identifying "Best"

- `metric_for_best_model="eval_loss"`: I have told the `Trainer` to use the loss calculated on evaluation set as the "scorecard" for the model.
- `greater_is_better=False`: Since this is a "loss" metric, a lower number is better. The `Trainer` tracks the lowest `eval_loss` it has seen across all 50-step checkpoints.

3. The Final Selection

- `load_best_model_at_end=True`: This is the most critical flag. At the end of training session, the `Trainer` will look back at all the checkpoints it saved and it will reload the specific weights that achieved the lowest `eval_loss`.
- `save_total_limit=1`: To save space on drive, the `Trainer` will delete older, inferior checkpoints, keeping only the current best one and the most recent one.

4. Permanent Storage

- `trainer.save_model(OUTPUT_DIR)`: Once the training loop finishes and the "best" weights are reloaded into memory, this command writes the final LoRA adapters and configuration files to the directory.

Note: Total training time for 5100 samples was 17 hours, 32 minutes

D. Results:

I. Metrics used for evaluation:

- Rogue-1, Rogue-2, Rogue-L, BLEU, METEOR and strict exact match were used as metrics for evaluation. For computing 'Strict exact match', the ground truth answers and predicted answers were normalized using following dictionary. All these metrics follow the rule of 'the higher the better'.

```
# Map semantically equivalent answers to a canonical form

ANSWER_SYNONYMS = {

    "none":      ["none", "not present", "absent", "nothing", "zero",
"there are none", "no instruments"],

    "yes":       ["yes", "correct", "true", "present", "there is", "i can
see"],

    "no":        [ "false", "not", "none", "absent", "cannot see"],

    "normal":    ["normal", "no abnormality", "no findings",
"unremarkable"],

    "colonoscopy": ["colonoscopy", "colon", "colonoscopic"],

}
```

II. Baseline Model Performance

The baseline model was evaluated using the same system prompt on the same set of test samples. Performance comparison of baseline model with fine-tuned model is shown below in Table 1 and Table 2.

Table 1: Performance comparison of baseline model (no fine-tuning) vs fine-tuned model

Metric	Baseline	Fine-tuned
Strict Exact Match	0.1756	0.8216
ROUGE-1	0.2179	0.8742
ROUGE-2	0.0004	0.1792
ROUGE-L	0.2179	0.8717
BLEU	0.0075	0.2022
METEOR	0.0326	0.4993

Table 2: Performance comparison of baseline model (no fine-tuning) vs fine-tuned model as per source of questions

Source wise Breakdown:	Baseline (%)	Fine-tuned (%)
Esophagitis	0.0992	0.9072
Instrument	0.2447	0.7118
Normal	0.1514	1.0000
Polyps	0.2480	0.7268
Ulcerative Colitis	0.1553	0.9176

III. Fine-tuned Model Performance

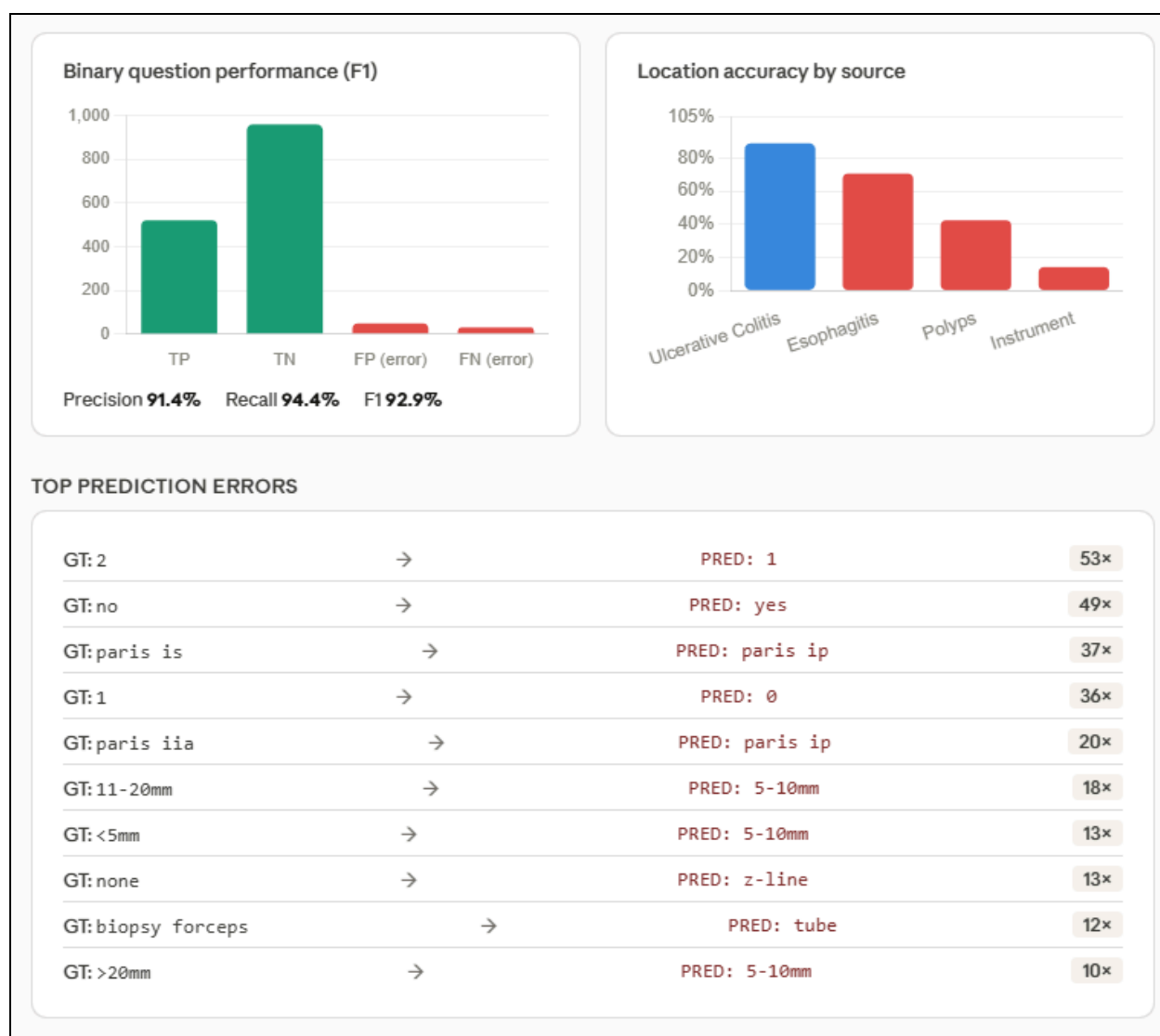


Figure 2: Performance analysis with respect to (A) Binary questions , (B) Category wise question (C) Questions with highest prediction errors

Here's a summary of the key insights from the evaluation:

Overall performance is solid at 82.2% exact match across 5,000 samples, with ROUGE-1 of 87.4% , the gap between them suggests some predictions are semantically close but not literally exact.

Biggest strength : binary (yes/no) questions: 94.1% accuracy with F1 of 92.9%. The model is slightly more prone to false positives (49) than false negatives (31), meaning it occasionally hallucinates findings that aren't there.

Biggest weakness : anatomical landmark localization: only 50.9% accuracy on "where in the image is the landmark?" questions. For Esophagitis specifically, landmark location accounts for 68 of its 131 total failures. This is the single most impactful gap.

Polyps is the hardest category (72.7%), with failures concentrated in four areas: polyp type classification (72×), color description (67×), location (62×), and size grading (56×). These require fine-grained visual reasoning the model struggles with.

Instrument spatial localization is nearly broken (14.3% on location questions). The model can often detect that an instrument is present but cannot reliably say where it is in the frame.

Paris classification confusion is a systematic error. The model collapses **paris is** and **paris iia** into **paris ip** (the most common class), suggesting it learned a frequency bias rather than visual discriminative features.

Polyp size grading bias: the model gravitates toward **5-10mm** regardless of actual size, misgrading both smaller (**<5mm** → **5-10mm**, 13×) and larger (**>20mm** → **5-10mm**, 10×) polyps in the same direction.

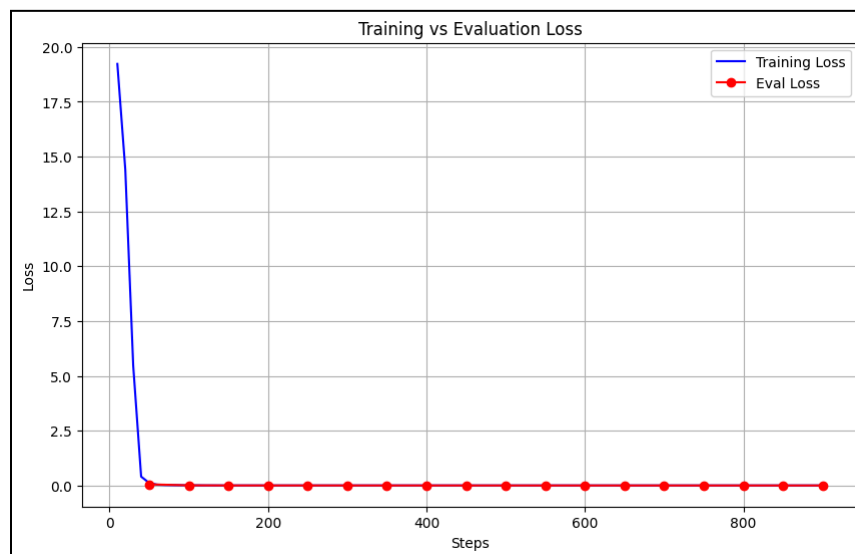


Figure 3: Training vs evaluation loss

E. Future Work

1. Effect of 'rank' parameter in QLORA algorithm on performance.
2. Use of other optimization techniques like sequential padding as discussed in libraries like '[unsloth](https://unsloth.ai/blog/nvidia-collab)' (<https://unsloth.ai/blog/nvidia-collab>)

Appendix:

1. PEFT (Parameter-Efficient Fine-Tuning)

It refers to any technique that allows you to fine-tune a massive model without updating all of its billions of parameters.

- **The Goal:** Reduce the massive computational and memory requirements of traditional "full fine-tuning."
- **The Analogy:** Instead of rewriting an entire 500-page textbook, you just add a few sticky notes with corrections.

2. LoRA (Low-Rank Adaptation)

LoRA is a **specific method** within the PEFT family. Instead of changing the original weights of the model (which are stored in large matrices), LoRA adds two much smaller matrices (called "adapters") to the side.

- **Rank (r):** This defines how small those side matrices are (e.g., $r=32$ in my script).
- **Trainable Params:** During training, the original model weights are frozen (not updated). Only the tiny A and B matrices are updated.
- **Memory Impact:** This significantly reduces the VRAM needed because you only need to store gradients for the adapters, not the whole model.

3. QLoRA (Quantized LoRA)

QLoRA is an **even more optimized version** of LoRA. It combines LoRA with **Quantization** to shrink the memory footprint further.

- **4-bit Quantization:** It "compresses" the original weights from 16-bit down to 4-bit (using a format like NormalFloat4 or NF4).
 - **The Benefit:** This allows you to fit a massive model (like SmolVLM2) into a consumer GPU (like your RTX 5080).
 - **In my code:** The value for `BitsAndBytesConfig` where `load_in_4bit=True` is set.
-

Comparison Table

Feature	Full Fine-Tuning	LoRA (PEFT)	QLoRA (PEFT)
Model Weights	All are updated	Frozen	Frozen + Compressed (4-bit)
Trainable Parts	Entire Model	Tiny Adapters (\$A\$ and \$B\$)	Tiny Adapters (\$A\$ and \$B\$)
VRAM Usage	Extremely High	Low	Lowest
Hardware	Enterprise Clusters	Prosumer GPUs	Consumer GPUs (RTX 5080)

4. Different Performance metrics

a) ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE measures the overlap of n-grams (sequences of words) between the prediction and the ground truth.

- ROUGE-1: Measures the overlap of individual words (unigrams). This is highly relevant for your "short phrase" endoscopy answers.
- ROUGE-2: Measures the overlap of word pairs (bigrams).
- ROUGE-L: Measures the Longest Common Subsequence (LCS). It is effective for capturing the structure of a sentence without requiring exact consecutive matches.
- F-Measure: You are specifically collecting the fmeasure, which is the harmonic mean of precision and recall.

b) BLEU (Bilingual Evaluation Understudy)

Originally designed for machine translation, BLEU calculates a score based on the average of modified n-gram precisions.

- SmoothingFunction: In my code, `method1` is used to prevent the score from dropping to zero when a prediction is very short or has no high-order n-gram matches.
- Theoretical Use: BLEU is "precision-oriented"—it penalizes the model for adding extra words that aren't in the ground truth.

c) METEOR (Metric for Evaluation of Translation with Explicit ORdering)

METEOR is often considered superior to BLEU for small datasets because it addresses some of BLEU's limitations.

- Synonyms and Stemming: Unlike BLEU, METEOR can match words based on their stems (e.g., "polyp" vs "polyps") or synonyms. This is useful in clinical contexts where pluralization or slight variations in terminology occur.

d) Strict Accuracy (Exact Match)

This is the most "binary" metric in the analysis.

- Normalization: By using `normalize_answer`, we are likely removing punctuation, extra whitespace, and converting text to lowercase before comparing.
 - Clinical Significance: For questions like "Is there an instrument?" (Answer: 'yes'/'no'), this is the only metric that truly matters.
-